

End-to-End AI Development: Learn, Build, and Deploy Intelligent Systems

Day 4 : Regression Algorithms



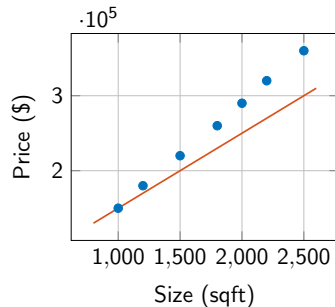
What is Regression?

Definition:

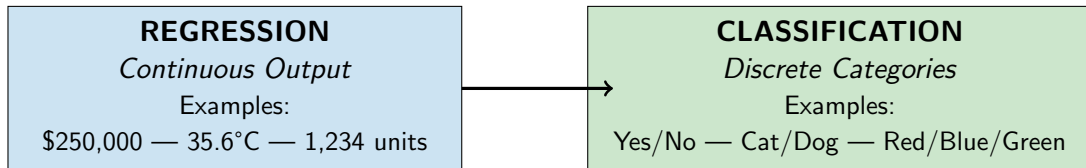
- Supervised learning technique
- Predicts **continuous** values
- Models relationship between variables
- Input: Features (X)
- Output: Continuous target (y)

Real-world Examples:

- House prices
- Stock prices
- Temperature forecasting
- Sales predictions



Regression vs Classification



Key Difference

Regression predicts **numbers on a continuous scale**, while classification predicts **discrete categories**.

Linear Regression: The Foundation

The Simple Model:

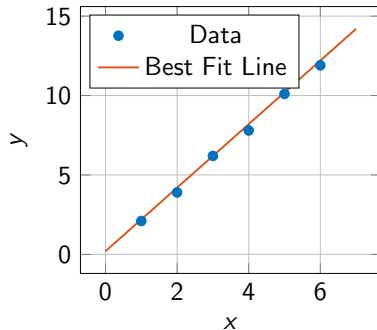
$$y = \beta_0 + \beta_1 x + \epsilon \quad (1)$$

Where:

- y = predicted value
- β_0 = intercept (bias)
- β_1 = slope (weight)
- x = input feature
- ϵ = error term

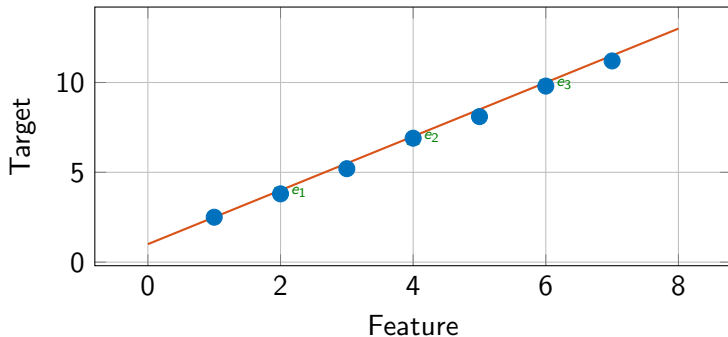
Multiple Features:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n \quad (2)$$



How Does It Work?

Goal: Find the best line that minimizes the error



Ordinary Least Squares (OLS)

Minimize the sum of **squared residuals**: $\sum_{i=1}^n (y_i - \hat{y}_i)^2$

Linear Regression: Python Implementation

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import numpy as np

# Sample data: house size vs price
X = np.array([[1000], [1500], [2000], [2500], [3000]])
y = np.array([150000, 200000, 250000, 300000, 350000])

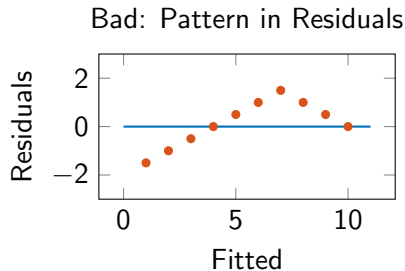
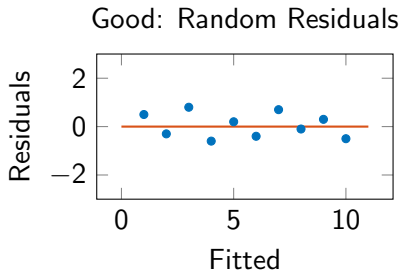
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train model
model = LinearRegression()
model.fit(X_train, y_train)

# Make predictions
predictions = model.predict(X_test)
```

Assumptions of Linear Regression

- 1 **Linearity**: Relationship between X and y is linear
- 2 **Independence**: Observations are independent
- 3 **Homoscedasticity**: Constant variance of errors
- 4 **Normality**: Errors are normally distributed
- 5 **No Multicollinearity**: Features are not highly correlated



When Linear Isn't Enough

The Problem:

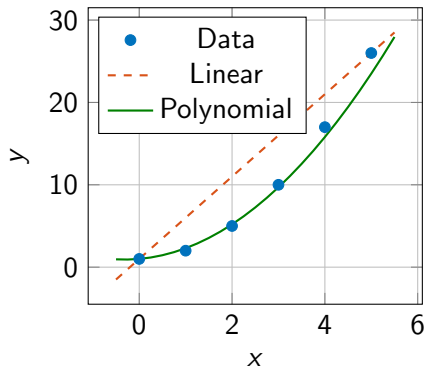
- Real data often has curves
- Linear models can't capture this
- Leads to **underfitting**

The Solution:

- Add polynomial terms
- $x^2, x^3, \dots, x^n$
- Still uses linear regression!

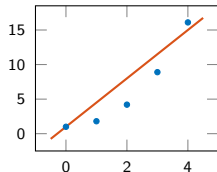
Polynomial Formula:

$$y = \beta_0 + \beta_1x + \beta_2x^2 + \dots + \beta_nx^n \quad (3)$$

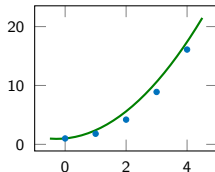


Degrees of Polynomial

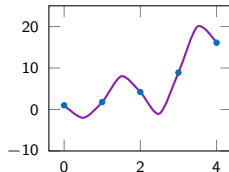
Degree 1 (Linear)



Degree 2 (Quadratic)



Degree 5 (Overfitting!)



Warning: The Goldilocks Problem

- Too low degree → **Underfitting** (too simple)
- Too high degree → **Overfitting** (memorizes noise)
- Just right degree → **Good generalization**

Polynomial Regression: Implementation

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
import numpy as np

# Non-linear data
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 5, 10, 17, 26])

# Create polynomial features
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

# Train polynomial regression model
model = LinearRegression()
model.fit(X_poly, y)

# Predict
X_test = np.array([[2.5], [3.5]])
X_test_poly = poly.transform(X_test)
predictions = model.predict(X_test_poly)

print(f"Predictions: {predictions}")
```

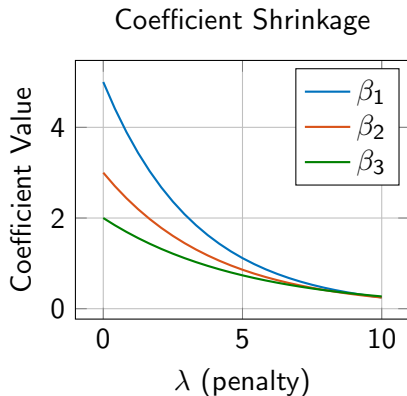
The Overfitting Problem

What happens with complex models?

- Large coefficients
- Model too sensitive to noise
- Poor generalization
- High variance

Ridge Regression Solution:

- Add penalty for large coefficients
- Shrink coefficients toward zero
- Better generalization
- Reduces overfitting



Standard Linear Regression Cost:

$$\text{Cost} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (4)$$

Ridge Regression Cost (with L2 penalty):

$$\text{Cost} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2 \quad (5)$$

Where:

- λ = regularization parameter (controls penalty strength)
- β_j = model coefficients
- Higher λ = stronger penalty = smaller coefficients

Key Point

Ridge regression never sets coefficients exactly to zero, just makes them smaller.

Ridge Regression: Implementation

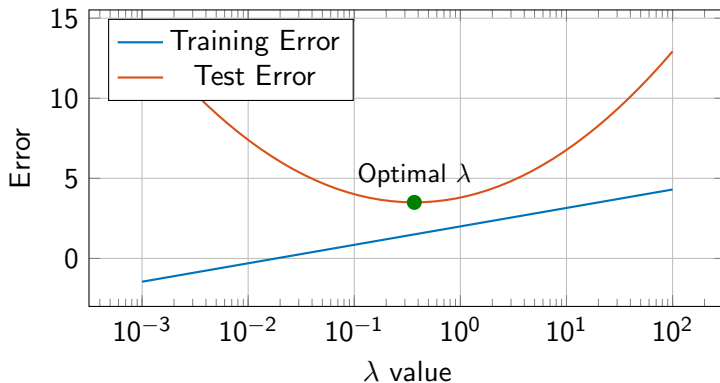
```
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# Create pipeline with scaling (important for Ridge!)
model = Pipeline([
    ('scaler', StandardScaler()), # Scale features first
    ('ridge', Ridge(alpha=1.0))   # alpha = lambda
])

# Train
model.fit(X_train, y_train)

# Predict
predictions = model.predict(X_test)
```

Choosing the Right λ (Alpha)



Finding Optimal λ

Use **Cross-Validation** to find the best value that minimizes test error.

Lasso Regression: Feature Selection

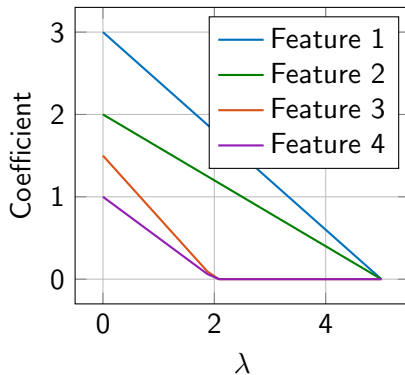
What makes Lasso special?

- Uses L1 penalty (absolute values)
- Can set coefficients to **exactly zero**
- Performs **automatic feature selection**
- Useful with many features

Lasso Cost Function:

$$\text{Cost} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j| \quad (6)$$

Lasso: Feature Elimination



Lasso Regression: Implementation

```
from sklearn.linear_model import Lasso, LassoCV

# Lasso with fixed alpha
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)

# See which features were kept
coefficients = lasso.coef_
print("Coefficients:", coefficients)
print("Non-zero features:", np.sum(coefficients != 0))

# Lasso with automatic alpha selection via CV
lasso_cv = LassoCV(cv=5, random_state=42)
lasso_cv.fit(X_train, y_train)

print(f"Best alpha: {lasso_cv.alpha_}")
print(f"R^2 score: {lasso_cv.score(X_test, y_test):.4f}")

# Feature importance (by coefficient magnitude)
feature_importance = np.abs(lasso_cv.coef_)
```

When to Use Ridge vs Lasso?

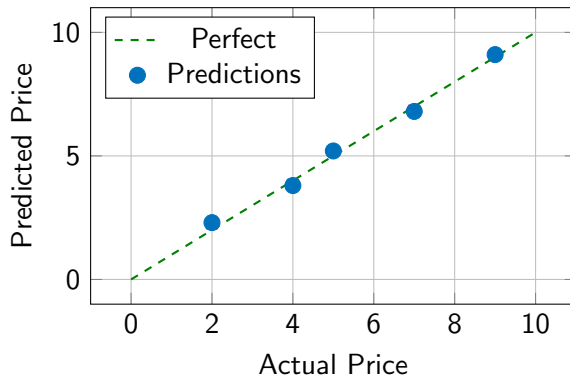
Aspect	Ridge (L2)	Lasso (L1)
Penalty	$\sum \beta_j^2$	$\sum \beta_j $
Coefficients	Shrinks to small values	Sets to exactly 0
Feature Selection	No	Yes (automatic)
Best When	All features relevant	Many irrelevant features
Multicollinearity	Handles well	Picks one randomly
Computation	Closed-form solution	Iterative

Elastic Net: Best of Both Worlds

Combines L1 + L2: $\lambda_1 \sum |\beta_j| + \lambda_2 \sum \beta_j^2$

How Good is Our Model?

We need metrics to measure prediction quality:



Mean Squared Error (MSE)

Formula:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (7)$$

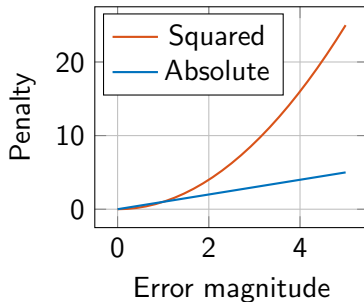
Properties:

- Squares the errors
- Penalizes large errors heavily
- Always positive
- Same units as y^2
- Lower is better

Example:

- Actual: [100, 200, 300] & Predicted: [110, 190, 310]
- Errors: [10, -10, 10]
- $\text{MSE} = \frac{10^2 + 10^2 + 10^2}{3} = 100$

Squared vs Absolute Error



Large errors get **heavily penalized!**

Root Mean Squared Error (RMSE)

Formula:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (8)$$

Why use RMSE?

- Same units as target variable
- Easier to interpret
- If predicting \$, RMSE in \$
- More intuitive than MSE

Example:

- $\text{MSE} = 100$
- $\text{RMSE} = \sqrt{100} = 10$
- Interpretation: "On average, predictions are off by \$10"

MSE vs RMSE Comparison

MSE	RMSE	Units
100	10	\$
10000	100	\$
0.25	0.5	meters

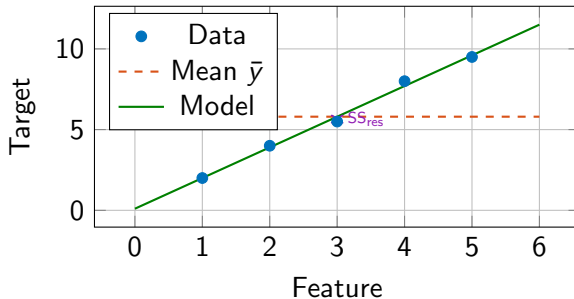
Note

RMSE is more sensitive to outliers than MAE (Mean Absolute Error)

R² Score (Coefficient of Determination)

Formula:

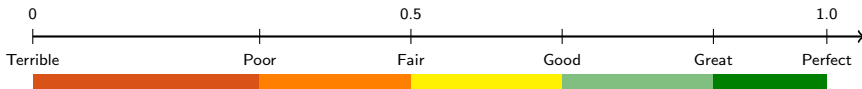
$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} \quad (9)$$



Understanding R^2 Score

Interpretation:

- Ranges from $-\infty$ to 1
- $R^2 = 1$: Perfect predictions
- $R^2 = 0$: Model = predicting mean
- $R^2 < 0$: Model worse than mean!



What's a Good R^2 Score?

- Depends on the problem domain
- Social sciences: 0.5 can be good
- Physical sciences: Often expect greater than 0.9
- Always compare to baseline!

Computing Metrics in Python

```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Make predictions
y_pred = model.predict(X_test)

# Calculate MSE
mse = mean_squared_error(y_test, y_pred)
print(f"MSE: {mse:.2f}")

# Calculate RMSE
rmse = np.sqrt(mse)
print(f"RMSE: {rmse:.2f}")

# Calculate R^2
r2 = r2_score(y_test, y_pred)
print(f"R^2 Score: {r2:.4f}")

# Mean Absolute Error (bonus)
from sklearn.metrics import mean_absolute_error
mae = mean_absolute_error(y_test, y_pred)
print(f"MAE: {mae:.2f}")
```

Best Practices for Regression

1. Data Preparation:

- Always explore data first
- Handle missing values appropriately
- Check for and handle outliers
- Visualize distributions and relationships

2. Feature Engineering:

- Use domain knowledge
- Create meaningful interactions
- Scale features for regularized models
- Remove highly correlated features

3. Model Selection:

- Start simple (Linear Regression)
- Use Ridge when all features matter
- Use Lasso for feature selection
- Try Elastic Net for best of both

Common Pitfalls to Avoid

1. Data Leakage

Never fit scaler or any transformer on the entire dataset! Only on training data.

2. Ignoring Assumptions

Check if linear regression assumptions hold. If violated, try transformations.

3. Overfitting

High training R^2 but low test R^2 ? Use regularization or reduce complexity.

4. Not Using Cross-Validation

Single train-test split can be misleading. Always use k-fold CV for robust evaluation.

5. Forgetting Feature Scaling

Ridge and Lasso require scaled features! StandardScaler is your friend.

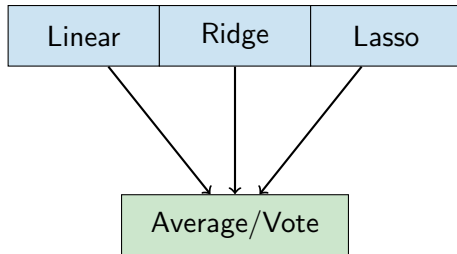
Evaluation Metrics Quick Reference

Metric	Formula	Range	Best Value
MSE	$\frac{1}{n} \sum (y_i - \hat{y}_i)^2$	$[0, \infty)$	0
RMSE	$\sqrt{\text{MSE}}$	$[0, \infty)$	0
MAE	$\frac{1}{n} \sum y_i - \hat{y}_i $	$[0, \infty)$	0
R ²	$1 - \frac{SS_{res}}{SS_{tot}}$	$(-\infty, 1]$	1
MAPE	$\frac{100}{n} \sum \frac{ y_i - \hat{y}_i }{ y_i }$	$[0, \infty)$	0

- **RMSE:** When large errors are especially bad
- **MAE:** When all errors weighted equally
- **R²:** For comparing models, understanding variance explained

Ensemble Methods Preview

Can we combine multiple models? Yes!



Final Prediction

Thank You For your Attention



Abdeldjalil Hani



Abdeldjalil Hani



@HaniAbdeldjalil



abdeldjalilhani0@gmail.com



+213 794 880 551

